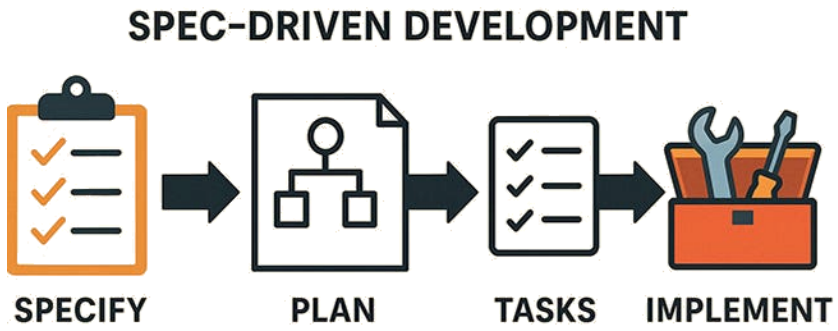


SPEC-DRIVEN DEVELOPMENT

Francisco Javier Moreno Diaz
fmorenod@gmail.com

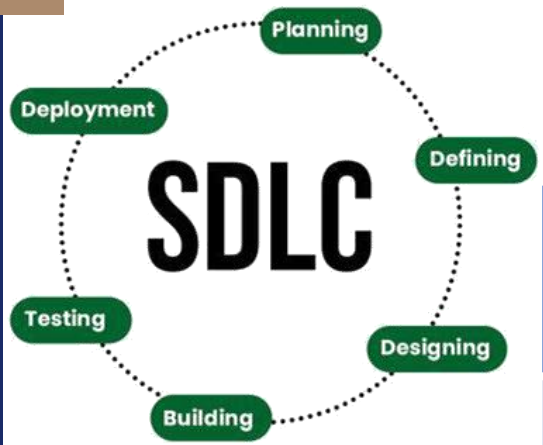
October-2026

Agenda



- What is SDLC ?
- Needs to define..
- Approaches for SDD
- Kiro
- Kiro on Claude Code
- SpecKit
- SpecKit Commands
- Learned Lessons
- References

What is SDLC ?



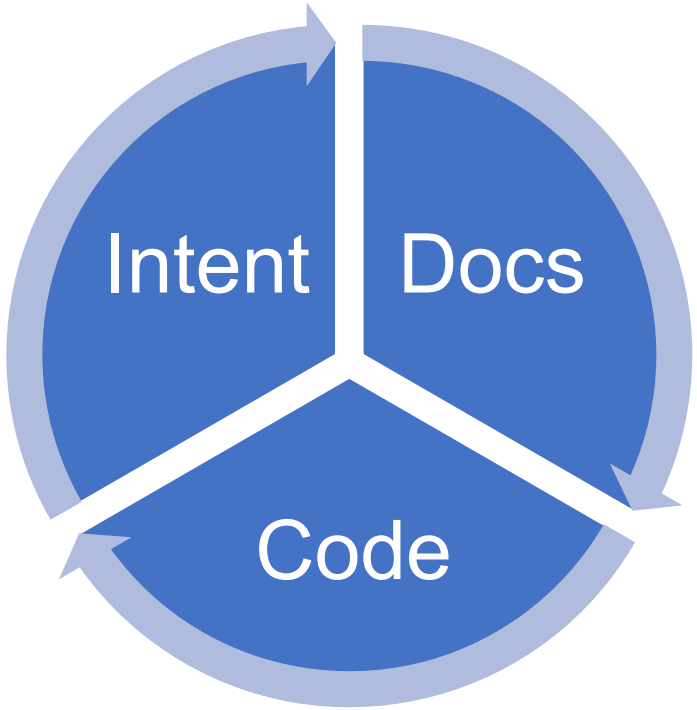
Feature	Waterfall	Agile	Vibe Coding (Chat AI)	Spec-Drive Development (SDD)
Role of Doc	Rigid Contract	Minimal Guidance	N/A	Executable Blueprint (Living Doc)
Context Management	Contract	Human Memory	Token Window	Local Index & Spec Files
Source of Truth	Front Doc	Working Software		Living Specification
Repeatability	Components/Libraries	Manual	Hit or Miss	Deterministic Plans
Code Quality	Human PR Review	Human PR Review	Review?	Spec Validated

From Vibe Coding to SDD

What is the core value ? According to Agile Manifesto, working software over comprehensive doc.

SDD inverts this relationship: specifications serve as the authoritative contract of system intent, and executable code is derived, generated, or verified against them

What is your role ?



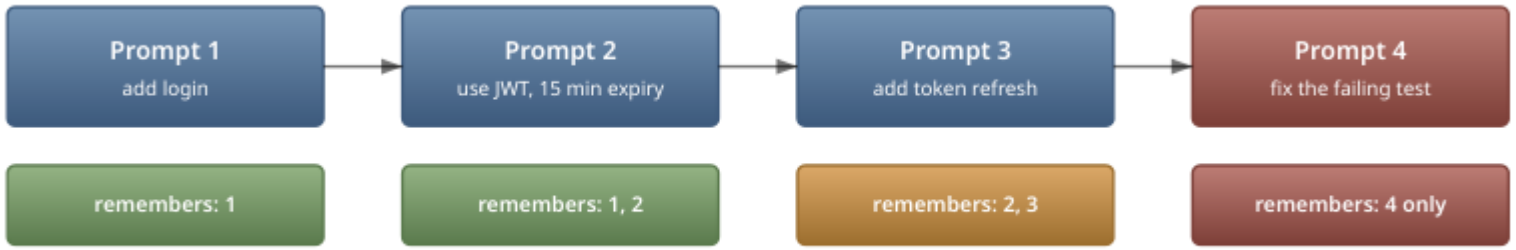
“Vibe coding optimizes for speed to first working version. SDD optimizes for total cost of ownership—including maintenance, iteration, and team coordination”.

“The biggest value out of this entire SDD process is context management. Literally that’s what it is. It is ensuring that the LLM, the tooling that you have, has the right context at the right time. That’s the magic of it.”

Vibe: Pros: Fast. Cons: no context and context drift, iterations, duplication and tech debt.
SDD: Pros: Spec as “superprompt”, validation step. Cons: Slower to read specs.

Vibe Coding	SDD
Chat iterativo	Contrato estructurado
Código como intención	Código como consecuencia
Esfuerzo en verificar	Esfuerzo en construir

«La IA no es el cuello de botella. Tu spec lo es.» — Addy Osmani



Agent context window: decisions slide out as the chat grows



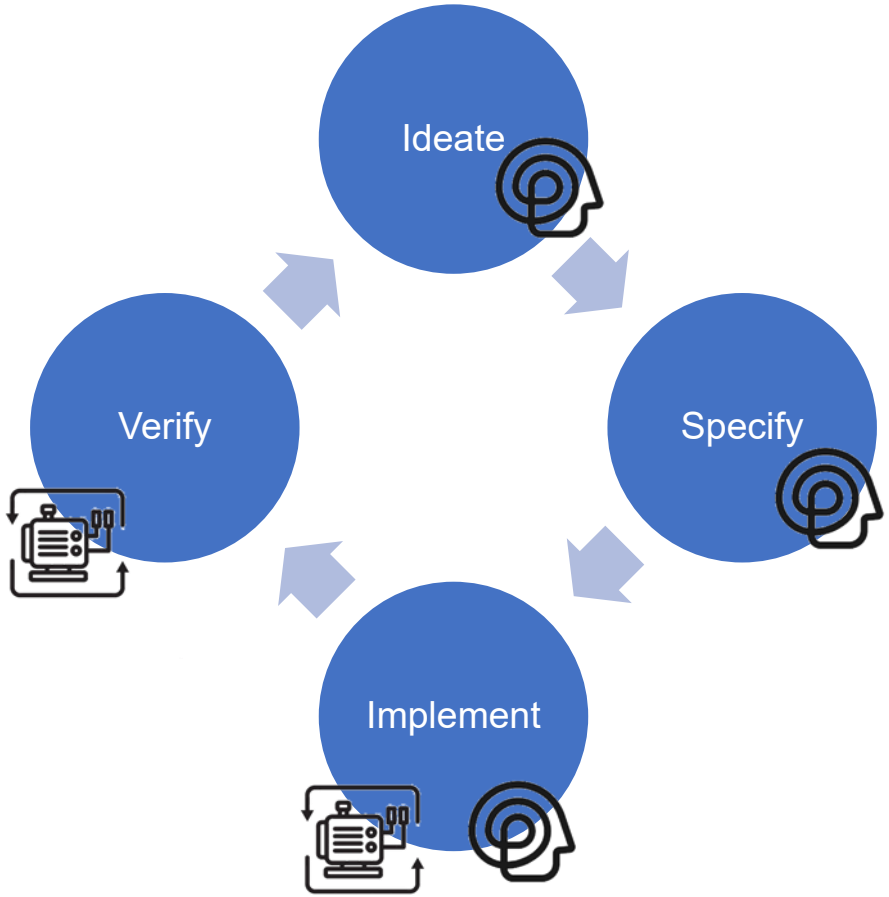
Imagen 3. Problemas estructurales del vibe coding

Not SDD for...

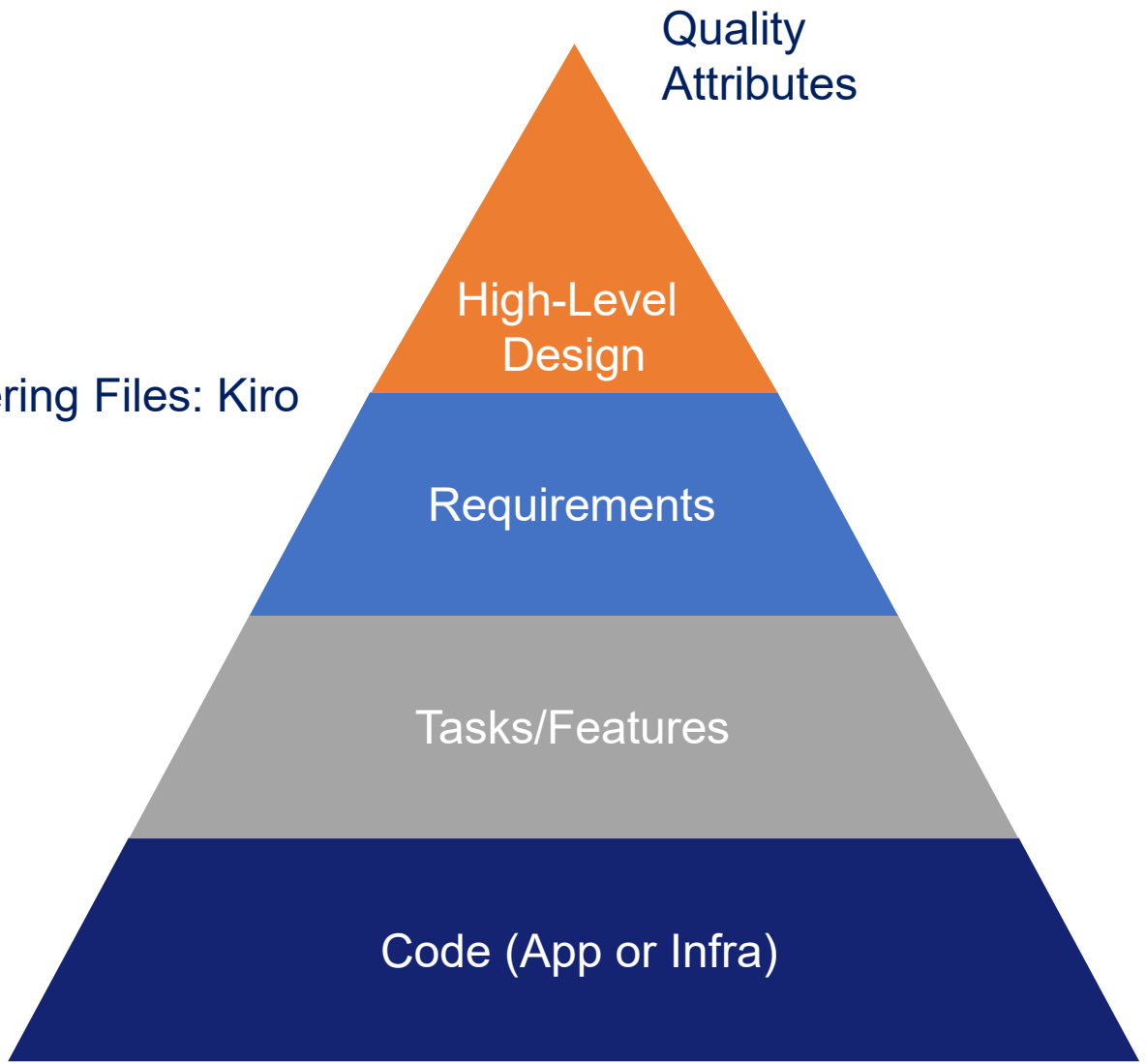
- SDD is not documenting after building.
- Throwaway Prototypes & Spikes: Exploratory code built to learn an API and discarded within hours.
- Emergency Production Hotfixes: Fix the incident directly in production first, then retroactively document the change in the spec.
- Simple CRUD Operations: Trivial changes where requirements are obvious and unstated assumptions are near zero.
- Myth: SDD is not writing exhaustive requirements in the style of specifications classics (public sector). << Public Sector
- Myth: SDD does not require abandoning agility.

Needs to define...

Objective: Define an organized method to manage the migration/reorg of workloads.



Steering Files: Kiro



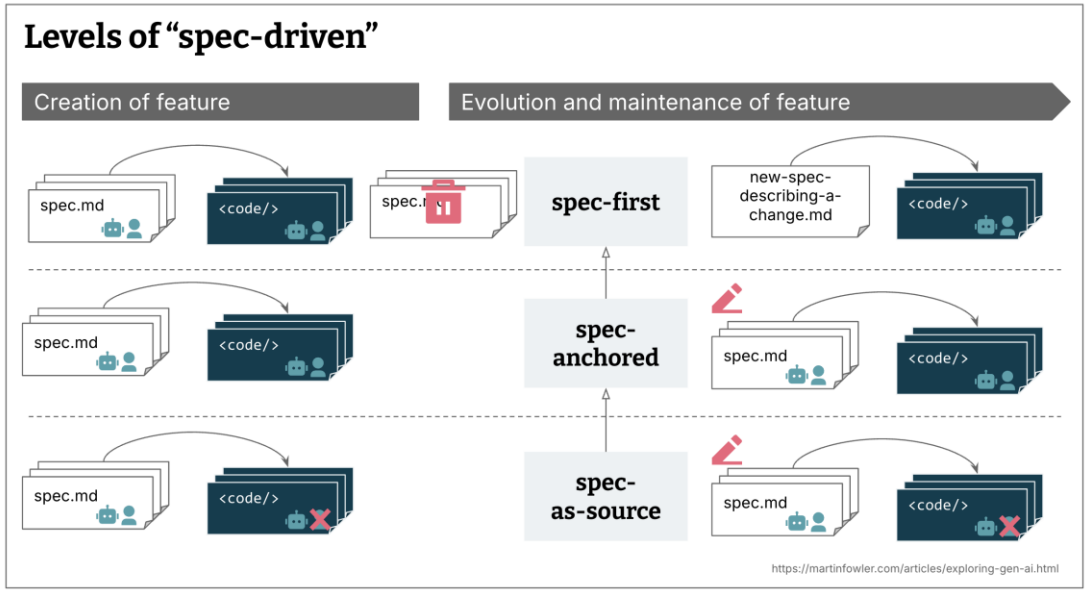
Approaches for SDD

Evolution of BDD and TDD

3 approaches:

- Spec-First
- Spec-Anchored
- Spec-as-source.

Differences due to modifications on initial specifications and code.
Structured similar: Memory bank, specs, features/tasks



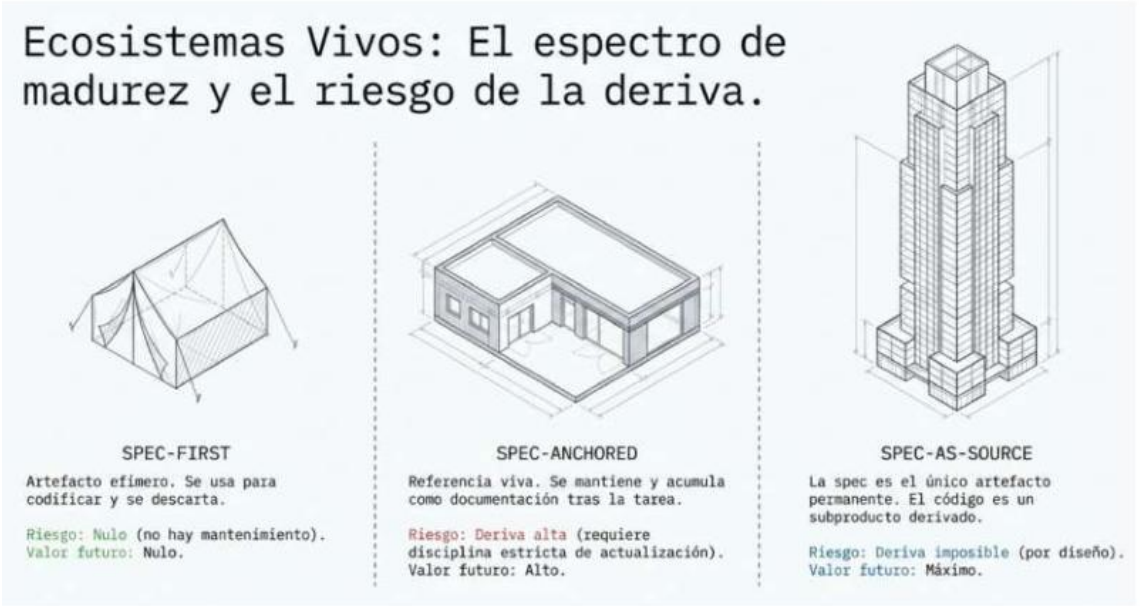
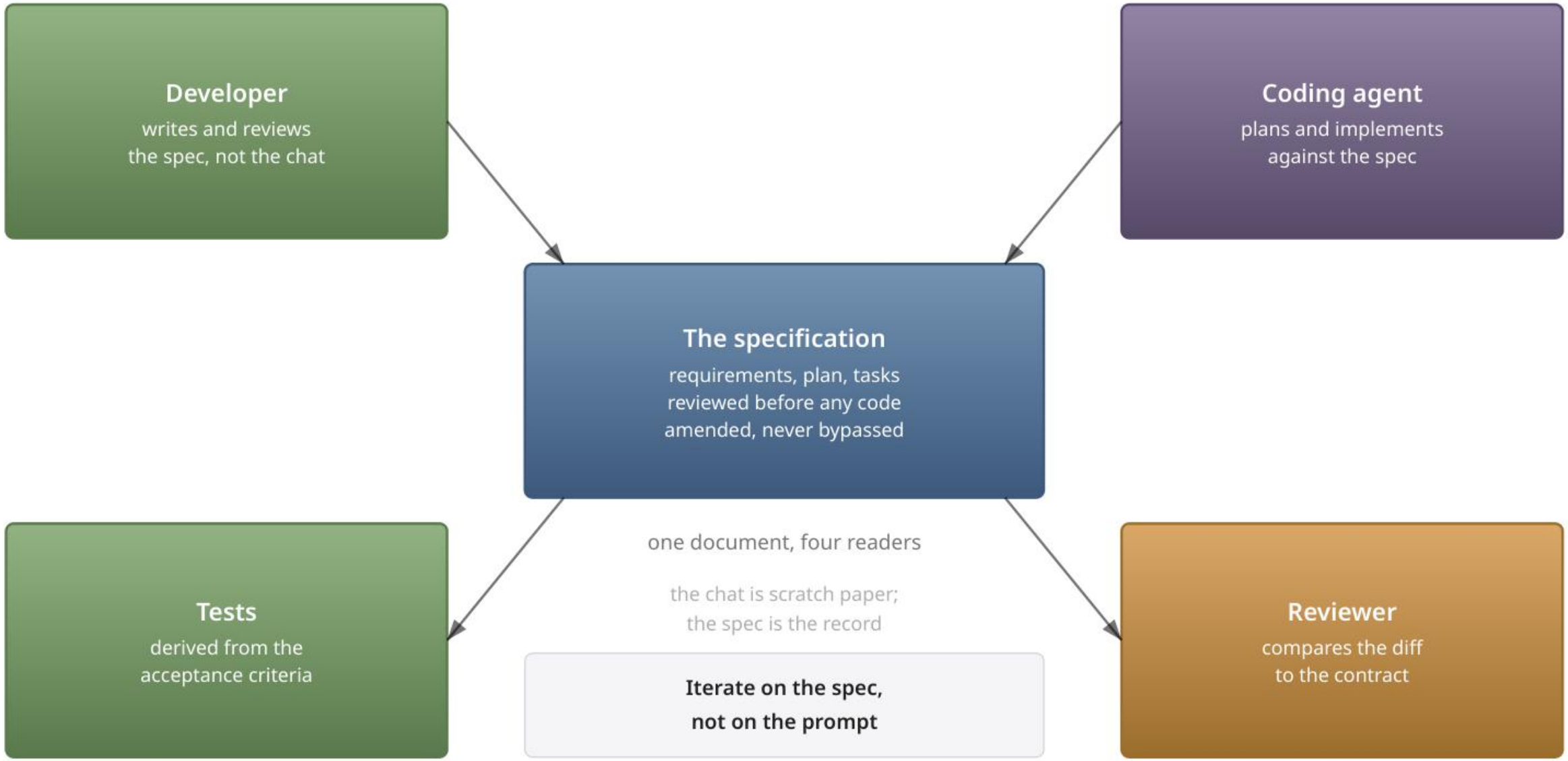


Imagen 15. Madurez y riesgo de deriva

The Core Idea



Maybe, BMAD is more according to having Personas/Agents to interact with more responsabilities

Short Comparison from Methodologies

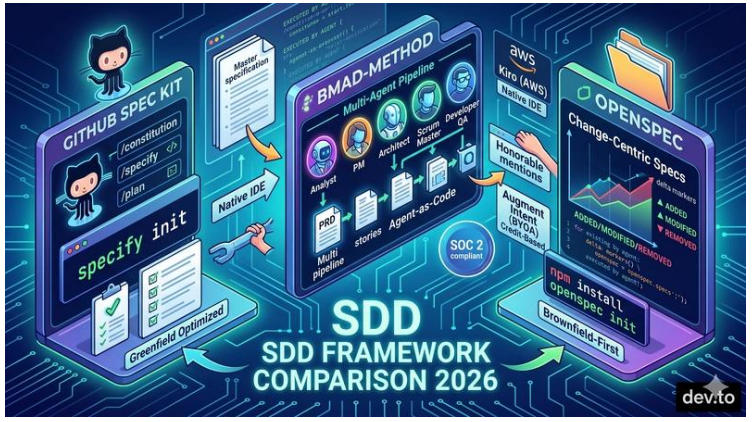
Framework	Primary Best Use Case	Architectural Model	Key Differentiators & Workflow
GitHub Spec Kit	Greenfield (0→1) new builds.	Feature-Centric Fragmentation: specs/NNN-feature/ directory containing separate spec, plan, and task files.	CLI (specify) with 4 linear phases (/speckit.specify → /speckit.plan → /speckit.tasks → /speckit.implement) governed by a global project Constitution (constitution.md).
OpenSpec (Fission AI)	Brownfield & legacy codebases.	Unified Source of Truth + Delta Specs: Changes written as diffs (ADDED, MODIFIED, REMOVED).	Action-based lifecycle (Propose → Apply → Archive) merging delta specs into a single living system document.
BMAD Method	Regulated Compliance (SOC 2, ISO, Finance).	Multi-Agent Persona Simulation: 12+ virtual team roles (Analyst, Architect, Dev, QA, PM).	Strict phase handoffs with explicit audit trails, digital sign-offs, and role-based validation matrices.
AWS Kiro	IDE-Native Greenfield development.	Integrated Workspace Artifacts: .kiro/specs/ and persistent .kiro/steering/ files.	VS Code fork (Code OSS) utilizing EARS notation for requirements, interactive approval gates, and background event hooks.

Summary Selection Quadrant

Building a new application from scratch? → GitHub Spec Kit or AWS Kiro (if AWS/IDE-integrated)

Modifying or extending a mature codebase? → OpenSpec.

Need audit trails, compliance, or multi-agent sign-offs? → BMAD Method.



From greenfield...



Imagen 6. Estructura del prompt



Imagen 8. Coste del error según la fase

Nivel: ALWAYS (Autonomía Total)	Acción: Ejecutar sin interrumpir ni preguntar.	Ejemplos: Ejecutar tests antes de commit, seguir nomenclatura, registrar errores.
Nivel: ASK FIRST (Punto de Control)	Acción: Proponer solución y y requerir aprobación humana.	Ejemplos: Modificar esquema de BD, añadir dependencias, cambiar API pública.
Nivel: NEVER (Línea Roja Absoluta)	Acción: Bloqueo estricto e inequívoco.	Ejemplos: Hacer commit de secretos, editar node_modules/, eliminar tests fallidos.

Imagen 12. Límites (Boundaries) de delegación

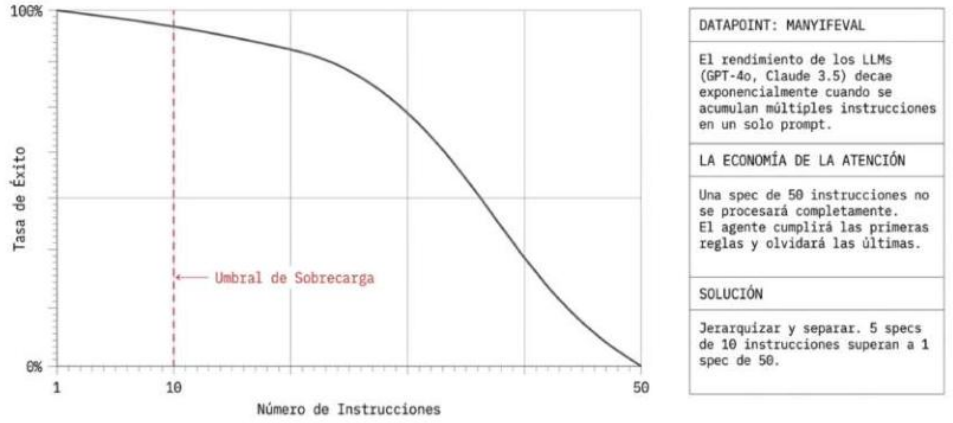


Imagen 13. La maldición de las instrucciones

It depends...

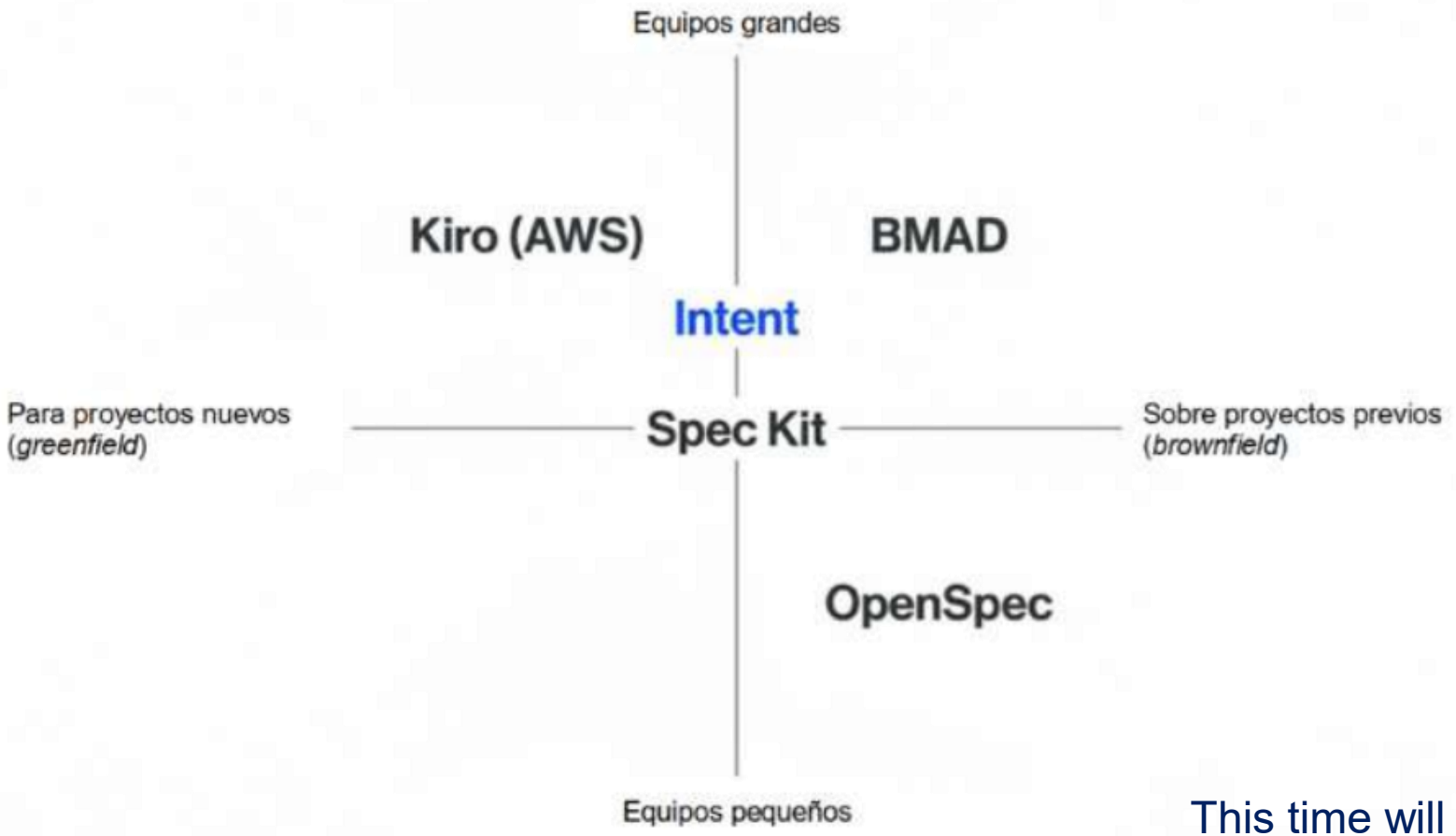
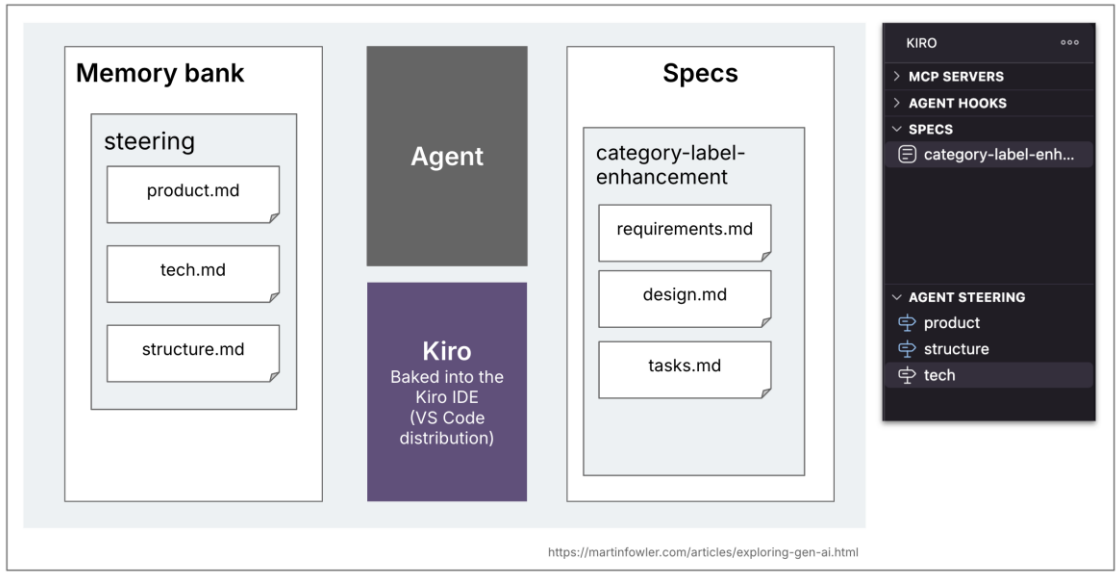


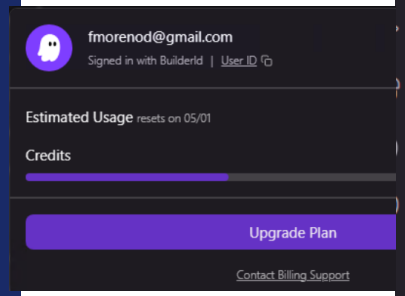
Imagen 22. Herramientas / idoneidad

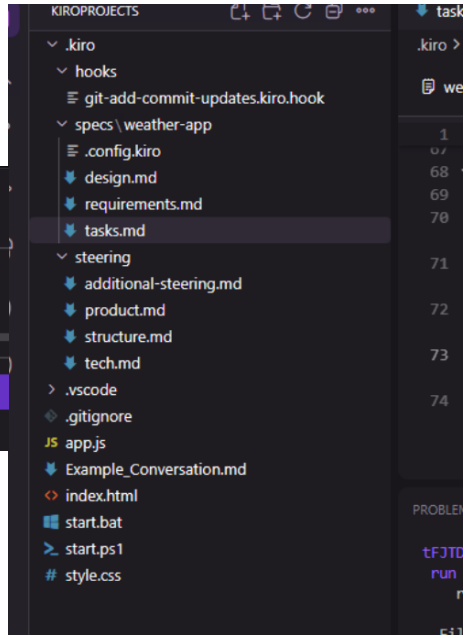
This time will review Kiro and SpecKit only.

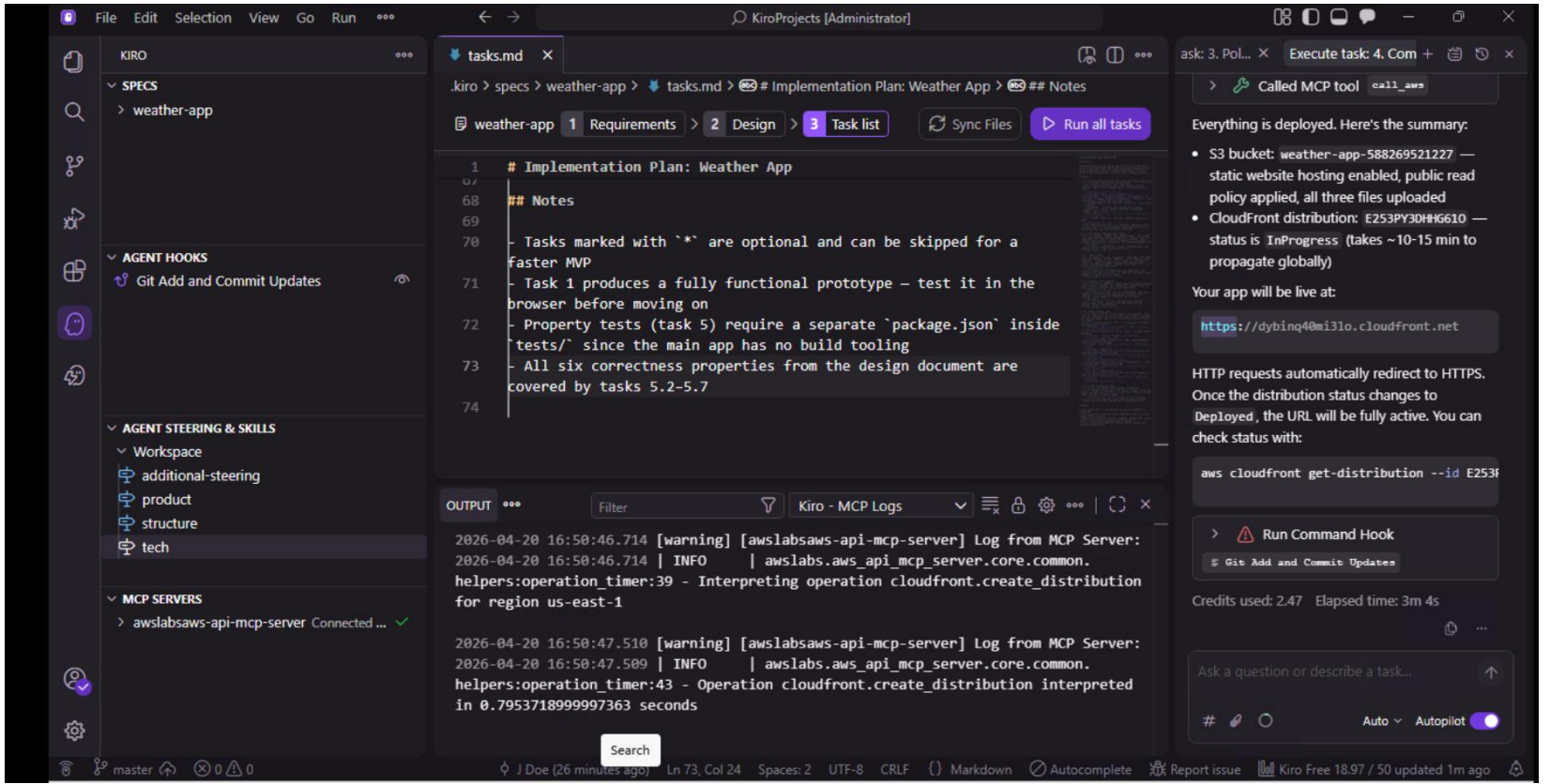
Kiro

- AI IDE with focus on Spec-Driven Dev, for features but support bug fixing.
- AWS managed and charge LLM.
- AWS Support and Training
- Simple and intuitive to learn

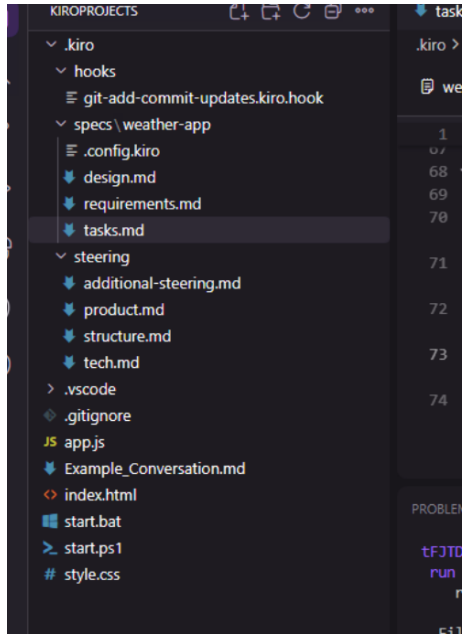








Kiro: Feature Trinity



Initial Prompt

Requirements

- Functional Requirements. Captures user intentions and its constraints. Defined what needs to do it in EARS syntax.

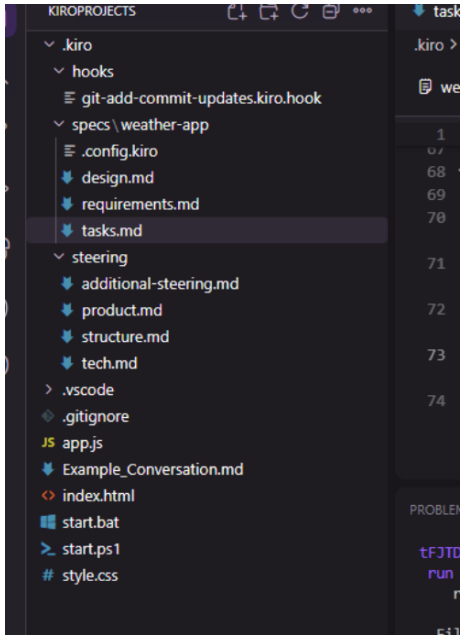
Design

- Schema abouts how to do it. State machines and relationships.

Tasks

- Executable plan. Atomic sequence plan that agent has to follow.

Kiro: Steering

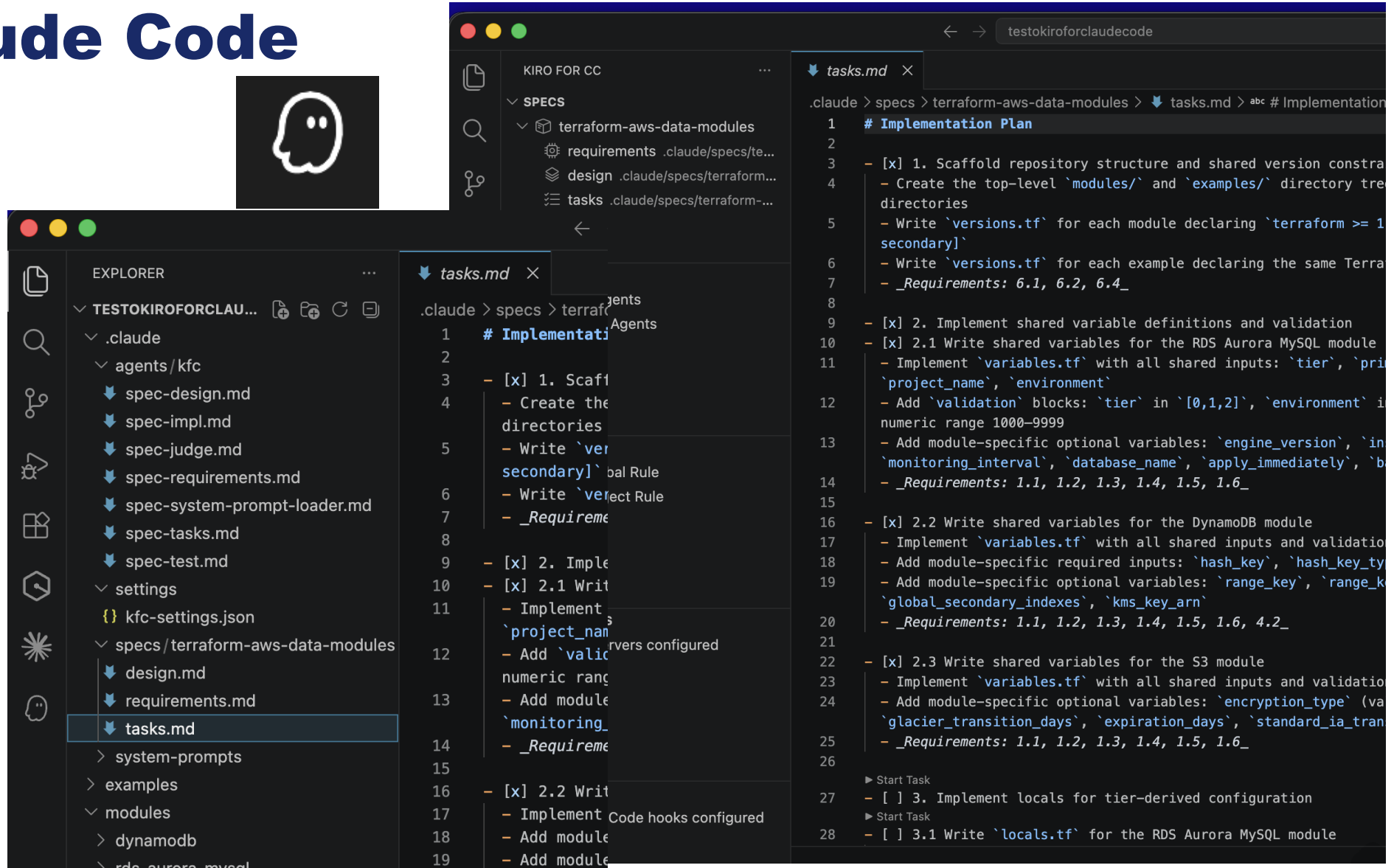


Define general rules that aren't link to the feature directly. There are examples for government and tech definitions, such as:

- Tech Stack: Frontend with React and Tailwind CSS. Backend with Python 3, FastAPI. DB with PostgreSQL and Prisma as ORM
- Patterns: Using Domain-Driven Development. All code has to have a header documentation about purpose of the file and methods.
- Naming definition: Use PascalCase for components names and camelCase for file names.

Kiro on Claude Code

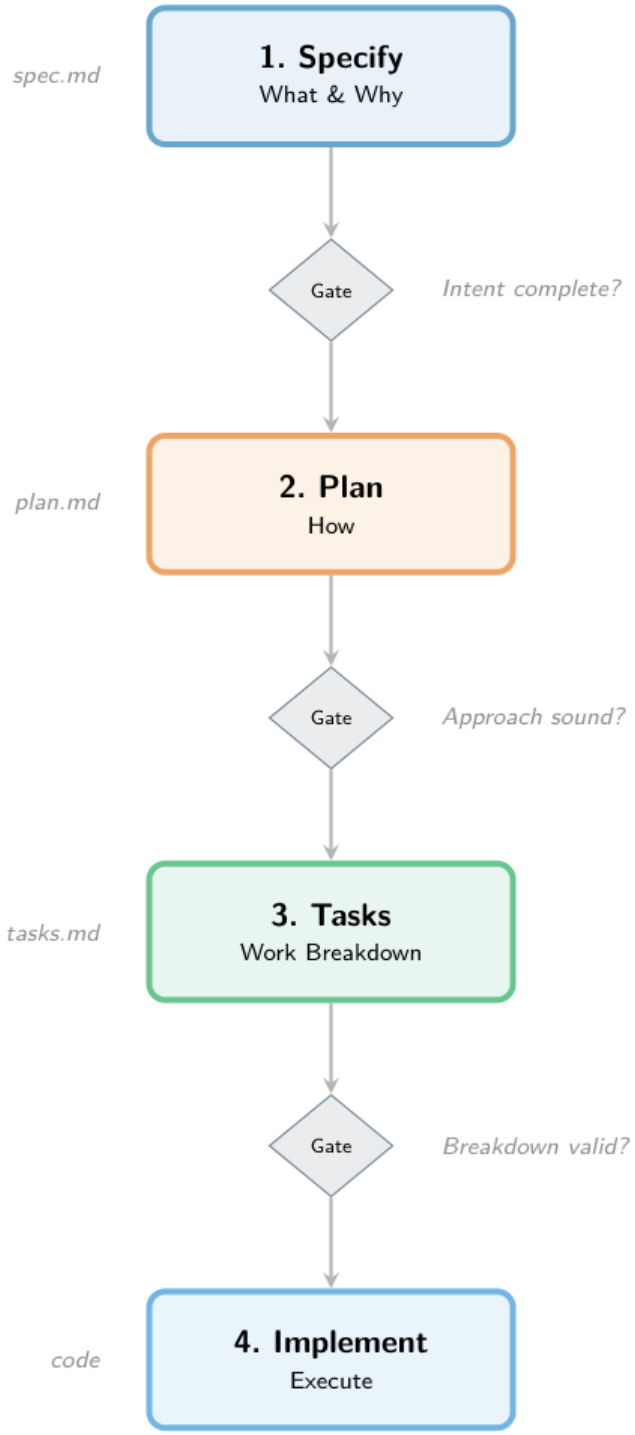
- VSCode Extension
- Need a Claude Code API Keys
- Bypass Security to act as agent
- Create more under-the-hood files (.claude/agents)



SpecKit

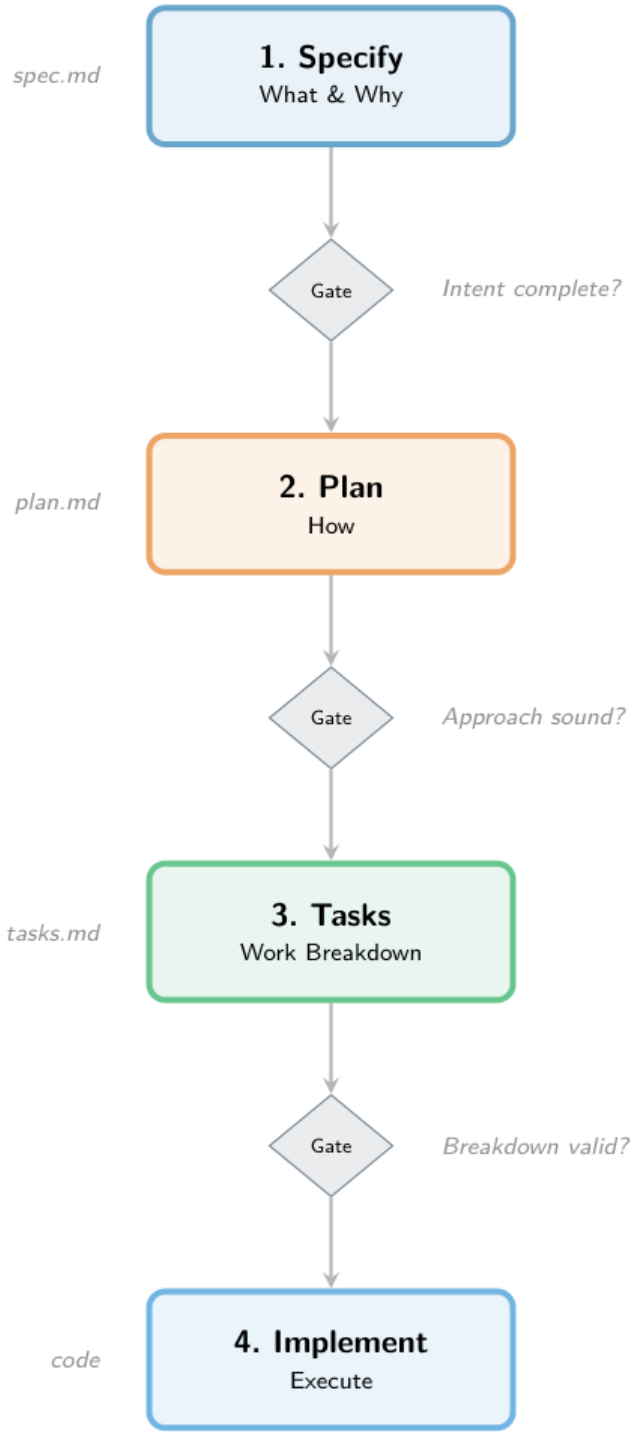
Installation SpecKit and define your LLM interaction (i.e. Claude Code, Gemini or Copilot)
Understand basics using official docs
Scaffolding your initial project using *specify init my-project --integration copilot*

```
my-project/
├── .specify/
│   ├── memory/
│   │   └── constitution.md        # [STATIC CONTEXT] Non-negotiable architectural
principles
│   ├── templates/                 # Markdown templates for spec, plan, tasks, and
checklists
│   ├── scripts/                   # Validation, git branch management, and gate
enforcement scripts
│   └── specs/                     # [DYNAMIC CONTEXT] Feature-specific specifications
│       └── 001-user-auth/
│           ├── spec.md            # What & Why (Business requirements & acceptance
criteria)
│           ├── plan.md            # How (Tech stack, architecture & contracts)
│           ├── tasks.md           # Numerical work breakdown (<200 lines/task)
│           ├── research.md        # Technical research notes
│           └── contracts/         # OpenAPI / AsyncAPI schemas
└── src/                           # Primary application source code
```



SpecKit

- **Phase 0: Constitution (/speckit.constitution):** Establishes immutable project rules (e.g., performance budgets, testing standards, security constraints)
- **Phase 1: Specify (/speckit.specify):** Defines functional behavior, user stories, and acceptance criteria in plain language without referencing technology choices **<< Input**
- **Phase 2: Plan (/speckit.plan):** Defines technical architecture, data models, dependencies, security controls, and API contracts based on the project Constitution **<< Input**
- **Phase 3: Tasks (/speckit.tasks):** Decomposes the technical plan into small, independent, numbered tasks (<200 changed lines each) with dependency markers and parallel flags. **>> Output**
- **Phase 4: Implement (/speckit.implement):** Executes tasks sequentially, running unit/integration tests after each task before marking it complete. **>> Output**



Core and Quality Commands

Core Command Sequence:

/speckit.constitution: Establishes immutable project rules (e.g., performance budgets, testing standards, security constraints)

/speckit.specify [feature summary]: Creates a numbered feature branch and generates spec.md

/speckit.plan [tech choices]: Generates plan.md, research.md, and contracts/

/speckit.tasks: Auto-generates dependency-ordered tasks in tasks.md.

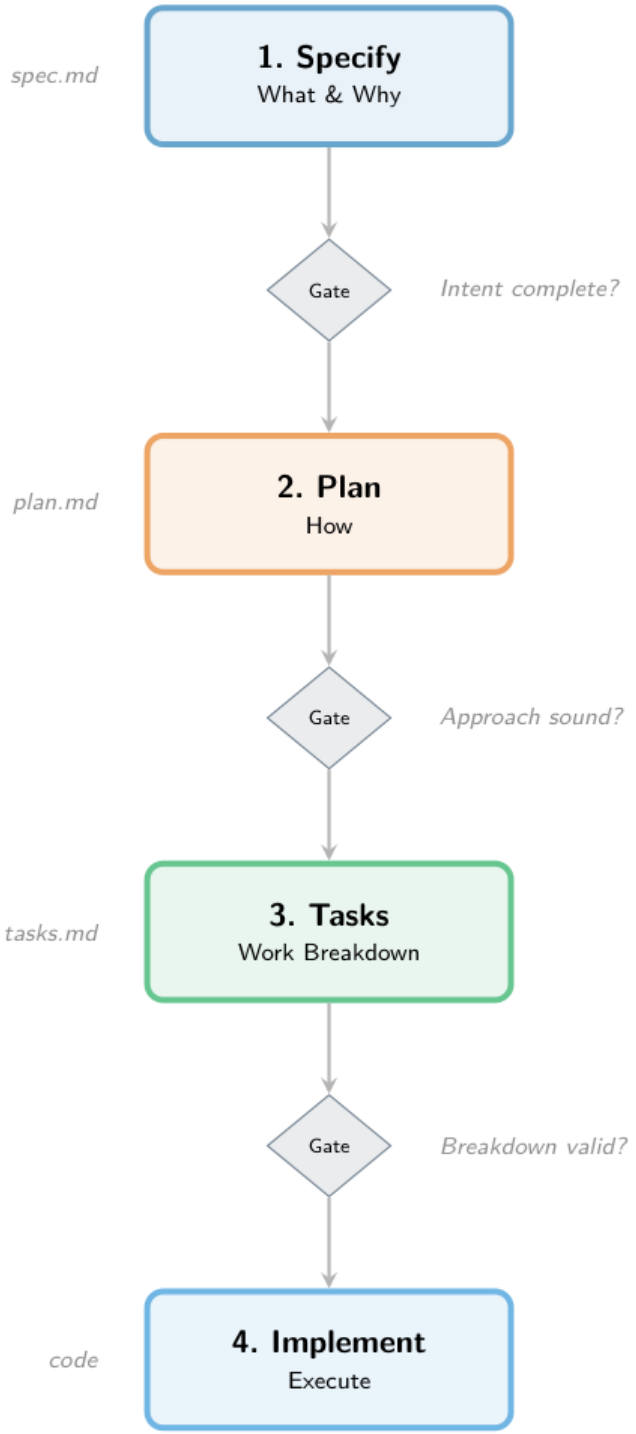
/speckit.implement: Executes tasks, generates code/tests, and updates task checkboxes.

Quality & Governance Commands:

/speckit.clarify: Conducts an interactive interview to resolve ambiguous requirements or open questions prior to technical planning.

/speckit.analyze: Cross-validates spec.md, plan.md, and tasks.md to output a coverage matrix identifying gaps, orphan tasks, and constitutional violations.

/speckit.checklist: Generates custom quality verification checklists ("unit tests for English") before code generation.

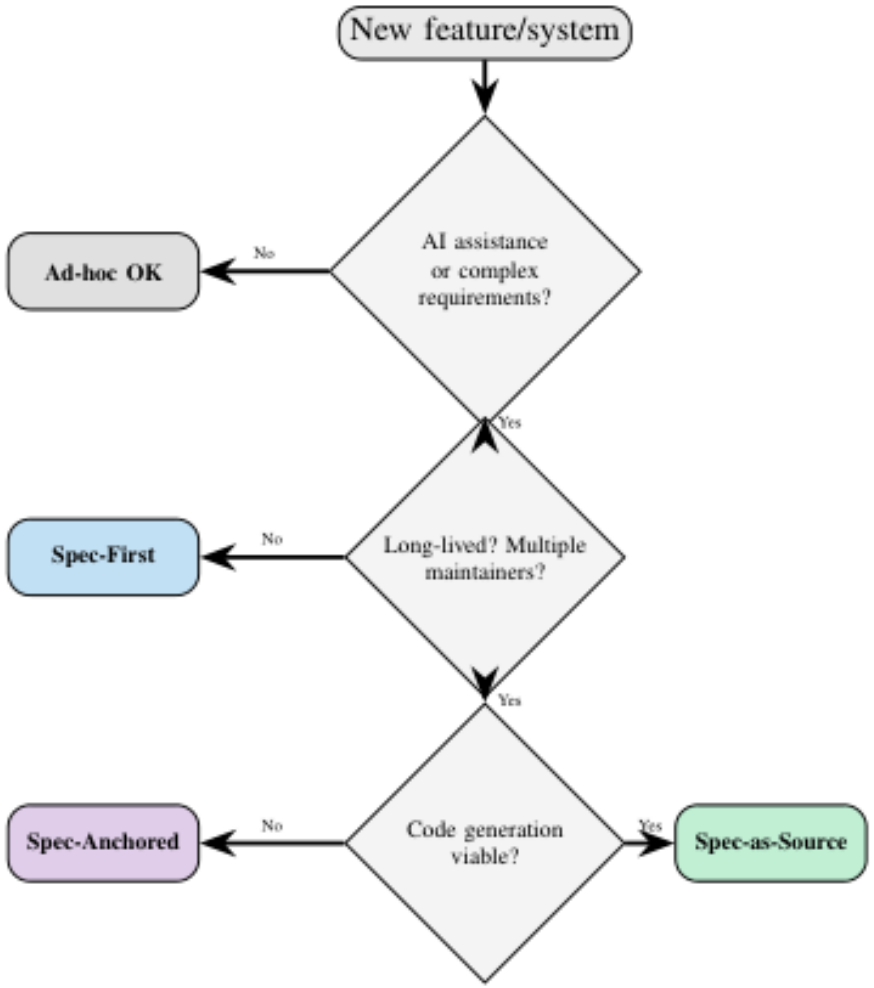


Out of scope of this ppt

My recommendations are:

- **Branching strategy:** Use gitflow for complex projects, otherwise use trunk-based strategy for simplicity.
- **Context Window:** Depends on scale and distribution of the project, it can be a problem. Try to reduce limit information on the trinity or use limited information, i.e. LLM Wiki and OpenViking
- **Deployment:** Use several type of testing on early steps -focus on integration-, to avoid rework on CI/CD pipelines.
- **Human on the loop:** Consequences of avoiding this step: Reputational damage, dismissal, legal issues, bankrupt, or layoffs.

Learned Lessons



- Follow a method to scale on a team and choose the right approach based on team' maturity and maintainability.
- Need of an orchestrator (stages and transversal policies) between projects (Pipeline).
- Difference between steering and structured files (design, requirements, tasks)
- Allow optional tasks for testing (Validation tasks are longer) or using external tools.
- Modifications has to be saved on documents or on steering to have single source of truth, otherwise you can save interactions on Export Conversations.
- Cost is based on the complexity and tier.

References

About Spec-Driven Dev:
arXiV (Cornell) ,
MartinFowler and
DeepLearning

About Kiro:
SkillBuilder Courses [1, 2]

About Claude Code
Anthropic Courses [A, B]

SDD for ScrumManager [C]

SDD with AI, Mark Veltzer [D]


Spec-Driven Development:
From Code to Contract in the Age of AI Coding Assistants

Deepak Babu Piskala
Seattle, USA
Technical Report

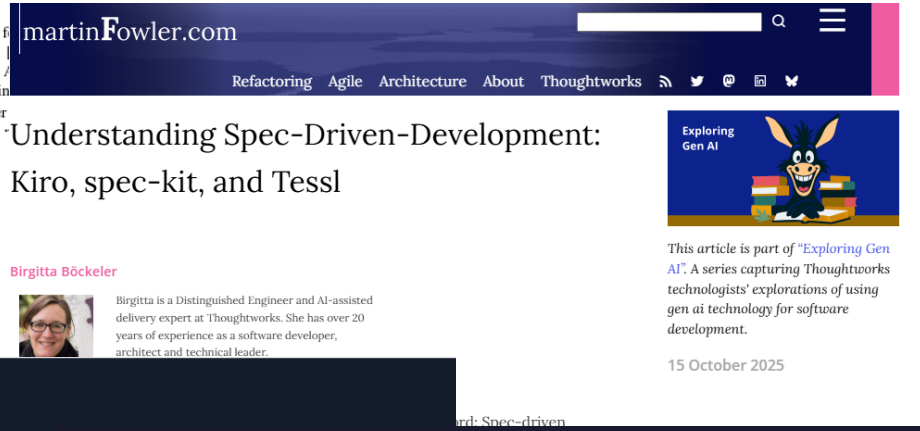
Abstract—The rise of AI coding assistants has reignited interest in an old idea: what if specifications—not code—were the primary artifact of software development? Spec-driven development (SDD) inverts the traditional workflow by treating specifications as the source of truth and code as a generated or verified secondary artifact. This paper provides practitioners




SDD – Spec Driven Development

Cuando el código es la consecuencia

Desarrollo dirigido por SDD en equipos ágiles con IA



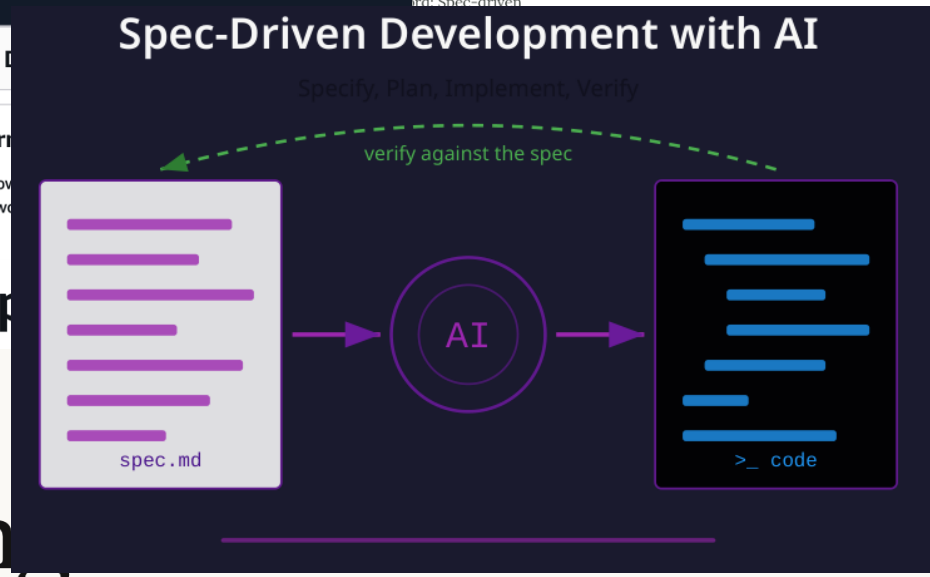
Understanding Spec-Driven-Development:
Kiro, spec-kit, and Tessl

Birgitta Böckeler

Birgitta is a Distinguished Engineer and AI-assisted delivery expert at Thoughtworks. She has over 20 years of experience as a software developer, architect and technical leader.

This article is part of "Exploring Gen AI". A series capturing Thoughtworks technologists' explorations of using gen ai technology for software development.

15 October 2025



Buildin

Activity

4, or at least, 2 Groups. You and your partner -optional- can go with one of them.

Group A:
Using Kiro and create a working architecture for a class using an Event-Driven Architecture.

Group B:
Using SpecKit and create a working architecture for a class using an Event-Driven Architecture.

Group C:
Using Kiro and create a working architecture for a class using an Microservices Architecture.

Group D:
Using SpecKit and create a working architecture for a class using an Event-Driven Architecture.

Limits:
Using Free/Paid resources for: AWS Resources, LLM, CLI.
Use Terraform to create the infrastructure.
Use your preferred language for the application (Java/Python/etc.) and containerize if necessary.

Activity

[5 mins] – Define the exam date and time, responsible for site.
[10 mins] - List your resources, limits, configurations (on paper). Understand the limits and architecture properties.
<Advices, experiences and solutions from the instructor>
[10 mins] - Configure Terraform, CLI –AWS/SpecKit/- or IDE –Kiro, VSCode and Plugin-
<2 expositions for students, max>
[20 mins] – Create the initial prompt. Review specifications and other documents: requirements, design, tasks.
[40 mins] – Start with the deployment.
[10 mins] – List your troubleshooting guide (on paper).
<2 expositions for students, max>
[10 mins] – Evaluate your instructor.
[10 mins] – **Sign** the attendance sheet

Before your sign:

- Review your previous attendance from last exam.
- Deliver the paper to the instructor.
- General Professor' evaluation (sigagestionhumana.unisabana.edu.co >> Autoservicio >> Gestion de Rendimiento >>Reportes Eval. Estudiantes>>Reportes de Evaluaciones de Estudiantes >> REPORTE AVANCE DE EVALUACIÓN) → Reach 80%

CLASES	GRUPO	TOTAL ESTUDIANTES	# DE ESTUDIANTES QUE HAN EVALUADO	% LOGRADO	# DE ESTUDIANTES QUE NO HAN EVALUADO
ARQUIT. EN LA NUBE DE AWS ACAD	1	28	7	25.00 %	21
ARQUIT. EN LA NUBE DE AWS ACAD	2	27	11	40.74 %	16

Advices and Experiences

Ud. No tiene claro las solicitudes (o cambian); o las limitaciones.
Ud. Puede interactuar con los archivos directamente, pero es mejor usar la herramienta CLI o el LLM.
Es un LLM, va a alucinar en algún momento.
Ud. No lee los documentos.
Piensan que las metodologías de SDD son iguales.